

Taller 4

Asignatura: Minería de Datos

Código: 2016325

Porcentaje: 25 %

Fecha limite de entrega: 30-07-2026

Modalidad: individual.

Horario de atención: Viernes 10:00-12:00.

Temas: Clases 20-24.

1. Objetivo

Ampliar la aplicación desarrollada en el Taller 2 mediante la incorporación de un sistema de búsqueda y recuperación de artículos científicos.

El sistema deberá permitir que un usuario escriba una consulta en lenguaje natural y obtenga una lista de artículos ordenados según su relevancia. Para construir esta funcionalidad, el estudiante deberá aplicar técnicas de procesamiento de texto, representación vectorial, recuperación de información, ranking y reducción de dimensionalidad.

El estudiante tendrá libertad para seleccionar los métodos que considere adecuados, siempre que documente, justifique y evalúe las decisiones tomadas.

2. Producto final esperado

El estudiante deberá ampliar la aplicación desarrollada en **Shiny** o **Streamlit** en el Taller 2, incorporando un nuevo módulo de búsqueda de artículos científicos.

No se espera construir una aplicación nueva. Se deberán conservar las funcionalidades desarrolladas anteriormente y agregar el sistema de recuperación de información.

2.1. Campo de búsqueda

La aplicación deberá incluir un campo de texto en el que el usuario pueda escribir una consulta en lenguaje natural.

Por ejemplo:

Aplicaciones de inteligencia artificial generativa en el diagnóstico de enfermedades.

La búsqueda deberá realizarse sobre el contenido textual disponible en la base SQLite, utilizando como mínimo:

- título del artículo;
- resumen;
- palabras clave, cuando estén disponibles.

El estudiante podrá incorporar otros campos si considera que aportan información relevante.

2.2. Resultados de la búsqueda

La aplicación deberá devolver una lista de artículos ordenados de mayor a menor relevancia.

Para cada resultado se deberá mostrar, como mínimo:

- posición en el ranking;
- título del artículo;
- autores;
- fecha de publicación;
- tema o categoría;
- DOI o enlace al artículo;
- puntaje de relevancia o similitud;
- resumen o fragmento de texto que permita comprender el resultado.

El usuario deberá poder seleccionar el número de resultados que desea visualizar, por ejemplo, los primeros 5, 10 o 20 artículos.

2.3. Comparación de estrategias

El estudiante deberá implementar y comparar al menos dos estrategias de recuperación de información.

Las estrategias pueden diferenciarse por:

- el procesamiento del texto;
- la representación vectorial;
- el método de reducción dimensional;
- el cálculo de similitud;
- el mecanismo de ranking;
- el tipo de recuperación: léxica, semántica o híbrida.

La aplicación deberá permitir seleccionar la estrategia que se desea utilizar o mostrar los resultados de ambas estrategias para una misma consulta.

No se establece un algoritmo obligatorio. Algunas comparaciones posibles son:

- TF-IDF y similitud coseno frente a LSA y similitud coseno;
- BM25 frente a LSA;
- BM25 frente a embeddings;
- TF-IDF frente a TF-IDF reducido mediante Truncated SVD;
- embeddings originales frente a embeddings reducidos;
- recuperación léxica frente a recuperación semántica.

2.4. Uso de reducción dimensional

El estudiante deberá aplicar al menos un método de reducción dimensional a la representación vectorial de los artículos.

La representación reducida deberá utilizarse para construir al menos uno de los rankings del buscador. No será suficiente calcular una reducción dimensional y mostrarla únicamente en el documento.

La consulta escrita por el usuario deberá proyectarse al mismo espacio reducido de los artículos antes de calcular su similitud.

El estudiante deberá indicar:

- dimensión original de la representación de los artículos;
- dimensión final después de la reducción;
- método utilizado;
- criterio empleado para seleccionar el número de componentes;
- información que intenta conservar;
- efecto de la reducción sobre el ranking, el tiempo de respuesta y el consumo de memoria.

Algunos métodos que pueden utilizarse son:

- PCA;
- Truncated SVD o LSA;
- NMF;
- ICA;
- autoencoders;
- otros métodos debidamente justificados.

En matrices dispersas de texto se recomienda considerar Truncated SVD, dado que puede aplicarse sin convertir toda la matriz a formato denso.

3. Ejemplo del resultado esperado

Suponga que el usuario escribe la siguiente consulta:

Aplicaciones de inteligencia artificial generativa en el diagnóstico de enfermedades.

La aplicación podría devolver un resultado como el siguiente:

Pos.	Artículo	Fragmento	Puntaje
1	Generative AI for clinical diagnosis	Uso de modelos generativos para apoyar decisiones clínicas...	0.87
2	Large language models in healthcare	Aplicaciones de modelos de lenguaje en sistemas hospitalarios...	0.79
3	Machine learning for disease prediction	Métodos predictivos para identificar enfermedades...	0.65

Los títulos y valores anteriores son únicamente ilustrativos. El puntaje dependerá de la estrategia utilizada y no necesariamente deberá interpretarse como una probabilidad.

Si el usuario cambia el método de recuperación, la aplicación deberá recalcular el ranking y podrá mostrar resultados diferentes.

4. Documentación del proceso

El estudiante deberá entregar un documento reproducible en formato `.ipynb`, `.Rmd` o `.qmd` en el que documente el proceso completo de construcción del sistema de recuperación de información.

El documento deberá contener el código ejecutado, los resultados obtenidos, la justificación de las decisiones metodológicas, la comparación entre las estrategias implementadas y la interpretación de los resultados.

El documento deberá permitir verificar cómo se pasó de los textos originales almacenados en SQLite a las representaciones vectoriales, las representaciones reducidas y los rankings utilizados por la aplicación.

4.1. Construcción del corpus

Documento:

- cuáles campos se utilizaron para representar cada artículo;
- número de artículos incluidos;
- cantidad de artículos sin título, resumen o palabras clave;
- tratamiento aplicado a los valores faltantes;
- idiomas encontrados en los documentos;
- criterios utilizados para incluir o excluir artículos.

No se permite reemplazar los textos faltantes por contenido inventado.

4.2. Procesamiento del texto

Documente qué tipo de procesamiento realizó y por qué.

Puede considerar:

- transformación a minúsculas;
- normalización de caracteres;
- tokenización;
- eliminación o conservación de stopwords;
- stemming;
- lematización;
- tratamiento de puntuación y números;
- conservación de términos científicos;
- construcción de unigramas, bigramas u otros n-gramas.

No es obligatorio aplicar todas las operaciones. El estudiante deberá justificar cuáles utilizó y cuáles decidió no utilizar.

4.3. Representación vectorial

Documente cuál o cuáles representaciones vectoriales utilizó y por qué son apropiadas para el problema.

Puede utilizar, entre otras:

- Bag of Words;
- TF-IDF;
- LSA;
- Word2Vec;
- GloVe;

- FastText;
- Doc2Vec;
- sentence embeddings;
- otras representaciones justificadas.

Debe indicar:

- dimensión de la representación;
- tamaño del vocabulario, cuando corresponda;
- si la matriz es dispersa o densa;
- recursos computacionales utilizados;
- ventajas y limitaciones de la representación seleccionada.

4.4. Tipo de recuperación

Documente si utilizó recuperación:

- léxica;
- semántica;
- híbrida.

Explique cómo el método determina que un artículo es relevante para una consulta.

Por ejemplo:

- BM25 considera coincidencia de términos, frecuencia y longitud del documento;
- TF-IDF con similitud coseno compara la orientación de los vectores;
- LSA compara artículos y consultas dentro de un espacio latente;
- los embeddings representan el significado del texto mediante vectores densos.

4.5. Reducción de dimensionalidad

Documento:

- método utilizado;
- dimensión original;
- dimensión reducida;
- criterio para escoger la dimensión final;
- procedimiento utilizado para proyectar la consulta;
- información conservada o perdida;
- efecto de la reducción sobre los resultados.

El estudiante deberá comparar el ranking producido con la representación reducida frente a otra estrategia de recuperación.

4.6. Mecanismo de ranking

Explique:

- cómo se calcula el puntaje de relevancia;
- cómo se ordenan los artículos;
- qué representa un puntaje alto;
- cómo se manejan los empates;
- si los puntajes de los métodos pueden compararse directamente;
- si se utilizaron citas, descargas o fecha como criterios secundarios.

No se considerará suficiente ordenar los artículos únicamente por fecha, citas o descargas.

4.7. Integración en la aplicación

Documente cómo integró el procesamiento y los modelos en Shiny o Streamlit.

Las estrategias presentadas y evaluadas en el documento reproducible deberán corresponder con las implementadas en la aplicación. No se aceptará documentar un método y utilizar otro diferente en el buscador sin explicación.

Las representaciones del corpus deberán calcularse previamente o almacenarse en caché. La aplicación no deberá reconstruir la matriz vectorial, entrenar el modelo de reducción o procesar nuevamente todo el corpus cada vez que el usuario realice una búsqueda.

5. Evaluación de los resultados

El estudiante deberá definir al menos cinco consultas relacionadas con el contenido real de la revista seleccionada.

Las consultas deberán incluir:

- una consulta con términos que aparezcan directamente en los artículos;
- una consulta expresada mediante términos relacionados o sinónimos;
- una consulta general;
- una consulta específica;
- una consulta en la que alguno de los métodos produzca resultados poco relevantes.

Para cada consulta, el estudiante deberá:

1. ejecutar las dos estrategias;
2. revisar los primeros resultados;
3. identificar cuáles artículos considera relevantes;
4. comparar las diferencias entre los rankings;
5. explicar cuál estrategia produjo mejores resultados y por qué;
6. analizar al menos un resultado incorrecto o poco relevante.

Como mínimo, deberá calcularse Precision@5 para cada consulta y estrategia:

$$\text{Precision@5} = \frac{\text{número de artículos relevantes entre los primeros 5}}{5}.$$

El estudiante podrá incluir otras métricas como MRR, Recall@k o nDCG.

6. Consideraciones computacionales

La solución deberá poder ejecutarse en un computador con un máximo de 16 GB de memoria RAM.

Para controlar el consumo de recursos se recomienda:

- mantener las matrices de texto en formato disperso;
- limitar justificadamente el tamaño del vocabulario;
- eliminar términos excesivamente frecuentes o poco frecuentes;
- evitar convertir matrices dispersas completas a matrices densas;
- calcular previamente las representaciones de los artículos;
- almacenar en caché los objetos necesarios;
- evitar entrenar modelos neuronales de gran tamaño;
- evitar depender de API de pago.

Los modelos preentrenados o embeddings son permitidos, pero no obligatorios.

Los objetos construidos previamente podrán guardarse en formatos apropiados para R o Python, por ejemplo:

- archivos `.rds` en R;
- archivos `.pkl`, `.joblib` o `.npz` en Python;
- archivos `.csv`, `.parquet` o tablas SQLite para datos tabulares.

7. Entrega

El Taller 4 tendrá dos productos principales:

1. Un documento reproducible en formato `.ipynb`, `.Rmd` o `.qmd`, en el que se documente y evalúe todo el proceso de preparación del texto, representación vectorial, reducción dimensional, recuperación de información y ranking.
2. La aplicación Shiny o Streamlit desarrollada en el Taller 2, ampliada con un buscador que implemente las estrategias desarrolladas en el documento reproducible.

Debe subir al **GitHub del curso**, mediante un **Pull Request**, dentro de la carpeta individual:

- documento principal `taller_4.ipynb`, `taller_4.Rmd` o `taller_4.qmd`;
- archivo `app.R` o archivos `ui.R` y `server.R` en el caso de Shiny;
- archivo `app.py` en el caso de Streamlit;
- base de datos SQLite utilizada;
- scripts auxiliares para procesamiento, representación vectorial, reducción dimensional, recuperación y ranking;
- objetos previamente contruidos que sean necesarios para ejecutar la aplicación;
- archivo con las consultas y resultados de la evaluación;
- archivo `README.md` con instrucciones de ejecución, estructura de los archivos y enlace de la aplicación desplegada;
- archivo de dependencias, por ejemplo `requirements.txt` o `renv.lock`.

El código deberá estar organizado y permitir reproducir los resultados.

8. Aclaraciones importantes

- No se considerará suficiente una búsqueda basada exclusivamente en filtros SQL, LIKE, expresiones regulares o coincidencia exacta de palabras.
- Estas herramientas pueden utilizarse como apoyo, pero el ranking principal deberá construirse a partir de una representación numérica del texto.

- El estudiante deberá implementar al menos dos estrategias de recuperación.
- Al menos uno de los rankings deberá utilizar una representación reducida.
- La reducción dimensional no podrá aparecer únicamente como un análisis aislado en el documento.
- No será obligatorio utilizar embeddings, redes neuronales o modelos de lenguaje.
- Los puntajes generados por métodos diferentes pueden encontrarse en escalas diferentes.
- Las funcionalidades desarrolladas en el Taller 2 deberán conservarse.
- No se deberán incluir contraseñas, tokens ni claves de API en GitHub.

9. Peso para cada punto

Criterio	Porcentaje
Funcionamiento e integración del buscador	20 %
Procesamiento y representación del texto	15 %
Recuperación de información y ranking	20 %
Reducción de dimensionalidad	20 %
Comparación y evaluación de resultados	15 %
Reproducibilidad, despliegue y documentación	10 %
Total	100 %